Manual técnico

- **Proyecto:** aceleración de una cadena de etapas bajo presupuesto (Subproyecto 3, primer problema).
- **Curso:** Análisis y Diseño de Algoritmos 2026-B, Universidad de Caldas.
- **Grupo 3:** Rafael Medina e Isabela Guerrero.
- **Repositorio:** https://github.com/Mandara2/ProyectoADAG3
- **Entrega:** viernes 6 de noviembre de 2026.

Este manual explica cómo está construida la solución, cómo se ejecuta, qué formatos usa, cómo se verificaron sus resultados y cuáles son sus límites.

Sobre los nombres. En este manual, *comprobación por fuerza bruta* es la verificación de la parte 1 que hace `src/comprobar.py`. La palabra *auditoría* se reserva para la segunda parte del Subproyecto 3, la cobertura mínima de auditoría, que no forma parte de esta entrega.

1. Qué resuelve

Una cadena tiene `n` etapas en serie. La etapa `i` dura `t_i`; acelerarla cuesta `c_i` y le quita `d_i` unidades de tiempo (`0 <= d_i <= t_i`). Con un presupuesto entero `B`, el programa elige qué etapas acelerar para que la cadena dure lo menos posible.

Como la duración sin acelerar, `Σ t_i`, es fija, el programa maximiza la reducción `Σ d_i` de las etapas elegidas, con la condición de que `Σ c_i <= B`. Es una mochila 0/1: el costo hace de peso y la reducción hace de valor.

2. Requisitos e instalación

- Python 3.10 o superior. Se probó con Python 3.12.10 en Windows 11.
- No usa librerías externas, así que no hay que instalar dependencias.

```
git clone https://github.com/Mandara2/ProyectoADAG3.git
cd ProyectoADAG3
```

Todos los comandos de este manual se ejecutan desde la raíz del repositorio.

3. Ejecución

Qué se quiere	Comando
Correr un solo caso, con su detalle completo	`python src/main.py ruta/al/caso.json`
Correr todos los casos de una carpeta	`python src/main.py pruebas/casos`
Lo mismo, con el detalle completo de cada caso	`python src/main.py pruebas/casos --detalle`
Elegir dónde guardar las salidas	`python src/main.py pruebas/casos otra_carpeta`
Comprobar un caso o una carpeta por fuerza bruta	`python src/comprobar.py pruebas/casos`
Correr las pruebas automáticas	`python -m unittest discover -s pruebas`
Generar un caso aleatorio	`python src/generar_casos.py <n> <semilla> [carpeta]`

Por defecto las salidas se guardan en `resultados/`. Por cada caso, el programa imprime una línea en pantalla:

```
ejemplo_tres_etapas: acelerar [1, 3], costo 9/10, duración 22 -> 15, 0.007 ms
```

Con un solo archivo, o con `--detalle`, después de esa línea se imprime el detalle del caso en cuatro bloques separados:

Bloque	Qué muestra
`[1] DATOS DEL CASO`	`n`, `B`, la duración sin acelerar y `t`, `c`, `d` de cada etapa
`[2] PROGRAMACIÓN DINÁMICA`	La tabla DP con el camino de la reconstrucción entre corchetes, la reconstrucción paso a paso y el resultado (sección 6.4)
`[3] COMPROBACIÓN POR FUERZA BRUTA`	Cada combinación revisada, sin usar la DP, y las soluciones óptimas (sección 6.5)
`[4] VEREDICTO`	`COINCIDE`, `NO COINCIDE` o `SIN COMPROBAR` (si el caso tiene más de 20 etapas)

Los bloques 3 y 4 también se guardan en `resultados/comprobacion/<id>.log`, y el resumen incluye el campo `coincide_con_fuerza_bruta`.

Si un caso es inválido, el programa lo reporta y sigue con los demás:

```
roto.json: INVÁLIDO, en la etapa 1 no se cumple d <= t (3 > 2)
```

4. Formato de entrada

Cada caso es un archivo `.json`:

```
{
  "id": "ejemplo_tres_etapas",
  "B": 10,
  "etapas": [
    { "t": 8, "c": 4, "d": 3 },
    { "t": 5, "c": 3, "d": 2 },
    { "t": 9, "c": 5, "d": 4 }
  ]
}
```

Campo	Tipo	Significado
`id`	texto	Nombre del caso. También es el nombre del archivo de salida.
`B`	entero `>= 0`	Presupuesto total.
`etapas`	lista	Una entrada por etapa, en el orden de la cadena.
`t`	entero `>= 0`	Duración normal de la etapa.
`c`	entero `>= 0`	Costo de acelerarla.
`d`	entero, `0 <= d <= t`	Tiempo que se reduce si se acelera.
`esperado`	opcional	Respuesta conocida. Solo la usan las pruebas automáticas; el programa la ignora.

Las etapas se numeran desde 1 según su posición en la lista.

Un caso se rechaza si: - No es JSON válido. - Le falta `id`, `B` o `etapas`. - `etapas` no es una lista, o una etapa no tiene `t`, `c` y `d`. - Algún número no es un entero mayor o igual a 0. - En alguna etapa `d > t`.

Sobre el caso del monitor. El monitor compartió un caso de referencia (`casos/entrada_G3_mochila_20_nodos.json`) en otro formato: trae, además de los datos de aceleración, datos de auditoría y de variantes de entregas posteriores. Como esta entrega cubre solo la aceleración, el programa usa el formato de esta sección y no lee ese archivo; si se le pasa, lo reporta como inválido.

5. Formato de salida

Por cada caso se crea `resultados/<id>.json`:

Campo	Significado
`id`, `n`	Nombre del caso y número de etapas
`tiempo_ms`	Tiempo promedio de 5 ejecuciones del algoritmo, en milisegundos. No incluye la lectura del archivo.
`etapas_aceleradas`	Etapas elegidas, en orden
`costo_usado`, `presupuesto`	Lo que se gastó y lo que había
`reduccion`	Tiempo total quitado: `DP[n][B]`
`duracion_original`, `duracion_final`	Duración antes y después de acelerar
`traza.casillas_tabla`	Tamaño de la tabla: `(n+1)(B+1)`
`traza.reconstruccion`	Una fila por etapa, de la última a la primera: `etapa`, `b`, `DP[i][b]`, `DP[i-1][b]`, `decision` y `presupuesto_restante`
`traza.tabla`	La tabla DP completa. Solo aparece si tiene menos de 500 casillas.

Además se crea `resultados/resumen.json`, con una fila por caso. Cuando se corre con detalle (un solo archivo o `--detalle`), cada fila incluye `coincide_con_fuerza_bruta`: `true`, `false` o `null` si el caso es demasiado grande para comprobarlo. Al correr un solo archivo, `resumen.json` queda solo con ese caso; para tenerlo completo hay que correr la carpeta.

Con detalle también se crea `resultados/comprobacion/<id>.log`, con el registro de la comprobación por fuerza bruta (sección 6.5).

Relación con la guía de la primera entrega. Para el G3, la guía pide entregar, por cada caso, el resultado, el tiempo y una traza con los estados y decisiones de la programación dinámica, las etapas aceleradas y los costos. Cada `resultados/<id>.json` cubre esos puntos así:

Lo que pide la guía	Campo de la salida
Resultado	`etapas_aceleradas`, `reduccion`, `duracion_original`, `duracion_final`
Tiempo por caso	`tiempo_ms`
Estados y decisiones de la DP	`traza.reconstruccion` y, si es pequeña, `traza.tabla`
Etapas aceleradas	`etapas_aceleradas`
Costos	`costo_usado` y `presupuesto`
Traza identificada por caso	El archivo se llama igual que el `id` del caso

La cobertura de auditoría que menciona la guía no aplica, porque esta entrega cubre solo el primer problema del Subproyecto 3. La sección 6 del informe muestra la traza del caso `ejemplo_tres_etapas` e indica dónde está la de cada caso.

6. Estructura del código

```
src/
├── lectura.py        lee y valida los casos
├── aceleracion.py    algoritmo: tabla, reconstrucción y traza
├── main.py           corre los casos, mide el tiempo y escribe las salidas
├── tabla.py          dibuja en la terminal la tabla DP y la reconstrucción
├── comprobar.py      comprueba por fuerza bruta y compara con la DP
└── generar_casos.py  crea casos aleatorios reproducibles
pruebas/
├── test_aceleracion.py
└── casos/            casos propios, con respuesta esperada cuando n <= 16
resultados/
├── <id>.json         resultado, tiempo y traza de cada caso
├── resumen.json      una fila por caso
└── comprobacion/     registro de la comprobación por fuerza bruta de cada caso
casos/                caso de referencia del monitor (otro formato; ver sección 4)
docs/
├── MANUAL_TECNICO.md  este manual
└── MANUAL_TECNICO.pdf
informe/
└── informe.tex       informe de la entrega
```

6.1 `lectura.py`

Es el único archivo que conoce el formato de entrada. Si los casos llegan en otro formato, solo cambia este archivo.

Función	Qué hace
`validar_caso(datos)`	Revisa un caso ya cargado y devuelve `(id, B, etapas)`. Lanza `CasoInvalido` con un mensaje claro si algo falla.
`leer_caso(ruta)`	Abre el archivo, lo convierte de JSON y llama a `validar_caso`.
`listar_casos(ruta)`	Si `ruta` es un archivo, devuelve solo ese; si es una carpeta, devuelve sus `.json` en orden alfabético.

6.2 `aceleracion.py`

Función	Qué hace	Costo
`llenar_tabla(etapas, B)`	Construye la tabla DP de `(n+1) x (B+1)`	`O(n·B)`
`reconstruir(dp, etapas, B)`	Recorre la tabla hacia atrás y decide qué etapas se aceleraron	`O(n)`
`resolver(etapas, B)`	Llama a las dos anteriores y arma el resultado con su traza	`O(n·B)`

6.3 `main.py`

1. Recibe un archivo o una carpeta.
2. Para cada caso: lo lee con `leer_caso`, ejecuta `resolver` 5 veces midiendo con `time.perf_counter()` y promedia el tiempo.
3. Escribe `resultados/<id>.json` e imprime la línea resumen.
4. Si es un solo archivo o se pidió `--detalle`, imprime los cuatro bloques con `tabla.py` y las funciones de `comprobar.py`, y guarda el registro de la comprobación.
5. Al final escribe `resultados/resumen.json`.

6.4 `tabla.py`

Dibuja en la terminal lo que hizo la DP con un caso: es el bloque 2 del detalle.

- **La tabla DP**, con una fila por etapa (etiquetada con su `c` y su `d`) y una columna por presupuesto `b`. Las casillas por las que pasa la reconstrucción van entre corchetes. Solo se dibuja si tiene hasta 31 columnas (`B <= 30`) y menos de 500 casillas; si no, se avisa y se omite.
- **La reconstrucción**, de la última etapa a la primera, con la decisión de cada paso.
- **El resultado** en una línea.

```
Etapas \ b   |  0   1   2   3   4   5   6   7   8   9  10
-----------------------------------------------------------
0 ninguna    |  0  [0]  0   0   0   0   0   0   0   0   0
1 (c=4, d=3) |  0   0   0   0   3  [3]  3   3   3   3   3
2 (c=3, d=2) |  0   0   0   2   3  [3]  3   5   5   5   5
3 (c=5, d=4) |  0   0   0   2   3   4   4   5   6   7  [7]
```

6.5 `comprobar.py`

Comprueba la DP de forma independiente: busca el óptimo sin usar la tabla, probando todas las combinaciones.

Función	Qué hace
`escribir_datos(B, etapas, escribir)`	Escribe los datos del caso (bloque 1 del detalle).
`fuerza_bruta(etapas, B, escribir)`	Prueba los `2^n` subconjuntos y registra cada uno. Devuelve la reducción máxima y todas las soluciones óptimas.
`comprobar(etapas, B, escribir)`	Llama a `fuerza_bruta` y escribe el óptimo (bloque 3). Devuelve `None` si el caso tiene más de 20 etapas.
`veredicto(dp, fuerza, escribir)`	Compara la DP con la fuerza bruta y escribe `COINCIDE`, `NO COINCIDE` o `SIN COMPROBAR` (bloque 4).
`comprobar_caso(id, B, etapas, escribir)`	Hace todo lo anterior para un caso; es lo que usa el comando `python src/comprobar.py`.

Para cada caso de hasta 20 etapas prueba los `2^n` subconjuntos en orden de tamaño. De cada uno registra:

- Qué etapas se aceleran y cuánto cuesta.
- Si cabe en el presupuesto. Si no cabe, lo anota y no lo evalúa.
- La reducción y la duración resultantes.
- Si es un nuevo mejor o empata con el mejor.

Al final lista todas las soluciones óptimas, corre la DP y escribe `COINCIDE` si la DP eligió una de ellas, o `NO COINCIDE` si no. El registro se imprime y se guarda en `resultados/comprobacion/<id>.log`. Con más de 1.024 combinaciones (más de 10 etapas) solo se escriben las que mejoran el resultado, para que el archivo no sea enorme. Termina con código 1 si algún caso no coincide.

Fragmento del registro de `ejemplo_tres_etapas`:

```
    #  Etapas aceleradas       Costo  Cabe Reducción Duración  Nota
    6  {1, 3}                      9  sí           7       15  nuevo mejor
    7  {2, 3}                      8  sí           6       16
    8  {1, 2, 3}                  12  no           -        -  no cabe: 12 > 10

Programación dinámica: acelera {1, 3}, reducción 7, costo 9.
Resultado: COINCIDE. La DP encontró una de las soluciones óptimas.
```

6.6 `generar_casos.py`

Crea un caso aleatorio a partir de `n` y una semilla. La misma semilla siempre produce el mismo caso. Cada etapa tiene `t` entre 1 y 20, `d` entre 0 y `t` y `c` entre 1 y 15. El presupuesto es la mitad de la suma de los costos, para que no todas las etapas quepan.

7. El algoritmo, paso a paso

`DP[i][b]` es la mayor reducción que se puede lograr usando solo las primeras `i` etapas y gastando como máximo `b`.

- `DP[0][b] = 0`: sin etapas no hay reducción.
- Si `c_i > b`, la etapa `i` no cabe: `DP[i][b] = DP[i-1][b]`.
- Si `c_i <= b`, se toma la mejor opción entre no acelerarla, `DP[i-1][b]`, y acelerarla, `DP[i-1][b - c_i] + d_i`.

La respuesta es `DP[n][B]`.

Reconstrucción. Se empieza en `i = n`, `b = B`. Si `DP[i][b]` es distinto de `DP[i-1][b]`, la etapa `i` se usó: se acelera y `b` baja en `c_i`. Si son iguales, no se aceleró. Luego se pasa a `i - 1`.

Ejemplo (`ejemplo_tres_etapas`, con `B = 10` y etapas `(t, c, d)`: `(8, 4, 3)`, `(5, 3, 2)` y `(9, 5, 4)`):

Etapa	b	DP[i][b]	DP[i-1][b]	Decisión	Presupuesto restante
3	10	7	5	acelerar	5
2	5	3	3	no acelerar	5
1	5	3	0	acelerar	1

Se aceleran las etapas 1 y 3, con costo 9. La reducción es 7 y la duración pasa de 22 a 15.

Por qué no hace falta guardar las decisiones. La comparación al llenar la tabla es estricta (`>`): si acelerar empata con no acelerar, se queda el valor de no acelerar. Por eso `DP[i][b] != DP[i-1][b]` ocurre exactamente cuando la etapa `i` se aceleró, y la reconstrucción no necesita una segunda tabla.

Memoria. La tabla completa ocupa `O(n·B)` enteros, y la reconstrucción la necesita entera. Una variante que solo guarda dos filas de valores y un bit por casilla con la decisión reduciría la memoria sin cambiar el orden `O(n·B)`; queda para la entrega de experimentación.

8. Casos de prueba propios

Están en `pruebas/casos/`. La respuesta esperada se calculó por fuerza bruta, probando todos los subconjuntos, sin usar la DP.

Caso	Qué prueba	Reducción esperada
`ejemplo_tres_etapas`	Caso básico con traza conocida	7
`ejemplo_almuerzo`	El presupuesto se usa completo	22
`voraz_por_razon_falla`	Elegir por mejor razón `d/c` daría 160; el óptimo es 220	220
`borde_presupuesto_cero`	`B = 0`	0
`borde_costo_cero`	Una etapa con costo 0 se acelera sin gastar	2
`borde_nada_cabe`	Ninguna etapa cabe en el presupuesto	0
`borde_todo_cabe`	Todas caben	11
`borde_sin_reduccion`	Una etapa con `d = 0`	4
`aleatorio_n15_s7`	Caso generado con 15 etapas	87
`aleatorio_n200_s1`	Caso grande: 200 etapas, `B = 824`	sin respuesta esperada; mide tiempo

9. Pruebas automáticas

`python -m unittest discover -s pruebas` corre 19 pruebas:

- Los ejemplos con resultado y traza conocidos.
- Los casos borde.
- Todos los casos de `pruebas/casos`, comparados con su campo `esperado`.
- 300 casos aleatorios con semilla fija, comparados contra fuerza bruta.
- La validación: `d > t`, presupuesto no entero, `etapas` que no es lista y JSON mal escrito.

- La comprobación por fuerza bruta: coincide con la DP en los casos propios, registra las 8 combinaciones del ejemplo y no intenta comprobar casos de más de 20 etapas.
- La vista de la tabla: marca con corchetes el camino de la reconstrucción y no dibuja tablas de más de 31 columnas.

10. Resultados verificados

Estos son los resultados de la versión entregada. Se obtienen con `python src/main.py pruebas/casos --detalle`, y cada uno se verificó de tres formas: contra su respuesta esperada, contra la comprobación por fuerza bruta del programa y contra una fuerza bruta escrita aparte.

Caso	n	B	Reducción	Esperada	Comprobación por fuerza bruta
`ejemplo_tres_etapas`	3	10	7	7	COINCIDE
`ejemplo_almuerzo`	3	5	22	22	COINCIDE
`voraz_por_razon_falla`	3	50	220	220	COINCIDE
`borde_presupuesto_cero`	2	0	0	0	COINCIDE
`borde_costo_cero`	2	0	2	2	COINCIDE
`borde_nada_cabe`	2	2	0	0	COINCIDE
`borde_todo_cabe`	3	100	11	11	COINCIDE
`borde_sin_reduccion`	2	5	4	4	COINCIDE (hay 2 soluciones óptimas)
`aleatorio_n15_s7`	15	33	87	87	COINCIDE (32.768 combinaciones)
`aleatorio_n200_s1`	200	824	942	—	SIN COMPROBAR (`2^200` combinaciones)

El caso de 200 etapas no se puede comprobar por fuerza bruta. Su resultado se verificó con una programación dinámica distinta, de una sola fila, que también da 942.

Para repetir la verificación:

```
python -m unittest discover -s pruebas
python src/main.py pruebas/casos --detalle
```

El primer comando debe terminar con `Ran 19 tests` y `OK`. El segundo debe mostrar 9 veces `Resultado: COINCIDE` y 1 vez `Resultado: SIN COMPROBAR`.

11. Limitaciones

- **Pseudo-polinomial:** el tiempo y la memoria crecen con el valor de `B`. Con un `B` de millones, la tabla tiene millones de casillas por etapa y puede no caber en memoria.
- Solo acepta valores enteros. Un `B` con decimales se rechaza.
- Si hay varias soluciones óptimas, devuelve una sola. Por cómo funciona la reconstrucción, cuando hay empate prefiere no acelerar la etapa de número mayor.

- La tabla completa solo se guarda en la traza si tiene menos de 500 casillas, para que los archivos de salida no sean enormes.
- La comprobación por fuerza bruta solo alcanza casos de hasta 20 etapas.
- No lee el formato del caso de referencia del monitor (sección 4).
- No incluye la segunda parte del Subproyecto 3, la cobertura mínima de auditoría, por indicación del curso.